SCHOOL OF COMPUTER SCIENCE, ENGINEERING AND APPLICATIONS

BHARATHIDASAN UNIVERSITY

TIRUCHIRAPPALLI 620 023

MASTER OF COMPUTER APPLICATION

NETWORK SECURITY LAB

SEMESTER – II

REGISTER NUMBER

2	5	M	C	A		

APRIL 2026

SCHOOL OF COMPUTER SCIENCE, ENGINEERING AND APPLICATIONS

BHARATHIDASAN UNIVERSITY

KHAJAMALAI CAMPUS

TIRUCHIRAPPALLI 620 023

CERTIFICATE

This is to certify that the bonafide record of work done in the School of Computer Science, Engineering and Applications, Khajamalai Campus, Bharathidasan University, Tiruchirappalli 620023, During the 2nd semester of the year 2025-2026.

SUBMITTED BY

NAME	
REG.NO	**25MCA**
CLASS	**MASTER OF COMPUTER APPLICATION**
SUBJECT	**NETWORK SECURITY LAB**
SUB.CODE	**MCA24207P**
SEMESTER	**II**

For the Practical Examination held on_____

Staff in-charge **Head of the Department**

1. External Examiner :

2. Internal Examiner :

CONTENTS

S.NO	DATE	TITLE	PAGE NO
1	06.02.2026	Write a Program to do Encryption and Decryption using Caesar Cipher	
2	06.02.2026	For a Given Text, Perform Encryption and Decryption using Data Encryption Standard	
3	10.02.2026	Write a Program to do Encryption and Decryption using Advanced Encryption Standard	
4	10.02.2026	For a Text of 30 Words, Perform Encryption and Decryption using RSA (Rivest–Shamir–Adleman Algorithm)	
5	13.02.2026	Write a Program to do Encryption and Decryption using Blowfish Algorithm	
6	20.02.2026	Write a Program that Performs Encryption and Decryption using Vigenere Cipher	
7	20.02.2026	Write a Program to do Encryption and Decryption using Digital Signature	
8	24.02.2026	For a Given Text, Perform Encryption and Decryption using Substitution Cipher	
9	24.02.2026	For a Given Numerical and Textual Data, Perform Web Security	

Ex.no: 1	Caesar Cipher
06.02.2026	

Aim: To develop a java program to do encryption and decryption using Caesar cipher

Caesar Cipher

The Caesar Cipher is a simple and one of the oldest encryption techniques in cryptography. It is a type of substitution cipher in which each letter in the plaintext is shifted by a fixed number of positions in the alphabet.

Procedure:

1. Define the class and main method as the entry point of execution.
2. Use Scanner (java.util package) to read input from the user.
3. Initialize a key value for shifting characters in the cipher.
4. Accept the plain text string and iterate through each character.
5. Perform encryption and decryption using Caesar Cipher formulas with modulo 26.
6. Store results in variables and display encrypted and decrypted output.

Java code for Caesar Cipher

```
import java.util.Scanner;

public class CaesarCipher {

public static voidScanner sc = new Scanner(System.in);

int key = 3;

System.out.print("Enter Plain Text: ");

String text = sc.nextLine();

String enc = "", dec = "";

for (int i = 0; i < text.length(); i++) {

char ch = text.charAt(i);

if (ch >= 'a' && ch <= 'z')

enc += (char) ((ch - 'a' + key) % 26 + 'a');

else

enc += ch;

}

for (int i = 0; i < enc.length(); i++) {
```

```
char ch = enc.charAt(i);
if (ch >= 'a' && ch <= 'z')
dec += (char) ((ch - 'a' - key + 26) % 26 + 'a');
else
dec += ch;
}
System.out.println("Encrypted: " + enc);
System.out.println("Decrypted: " + dec);
}
}
```

COMPILE AND RUN THE PROGRAM

```
>javac CaesarCipher.java
>java CaesarCipher
```

OUTPUT: CAESARCIPHER

```
Enter Plain Text: PremKumar
Encrypted: PuhpKxpdu
Decrypted: PremKumar
```

Result :

Encryption and Decryption using Caesar Cipher process is successfully executed.

Ex.no: 2	Data Encryption Standard
06.02.2026	

Aim: Develop a Given Text, Perform Encryption and Decryption using Data Encryption Standard

Data Encryption Standard

DES is an old encryption method that protects data by scrambling it using a secret key, but today it is mostly obsolete due to security weaknesses.

DES works by:

1. Dividing data into **64-bit blocks**
2. Applying **initial permutation**
3. Performing **16 rounds** of complex operations (substitution and permutation)
4. Using a **56-bit key** to transform data
5. Producing encrypted output (cipher text)

Procedure:

1. Define the class and main method to start execution and handle program flow.
2. Use Scanner (java.util package) to take key and plaintext input from the user.
3. Use DESKeySpec, SecretKeyFactory, and SecretKey (javax.crypto package) to generate the encryption key.
4. Create and configure Cipher (javax.crypto package) with DES/ECB/PKCS5Padding mode.
5. Perform encryption and decryption using cipher.init() and doFinal() methods.
6. Use Base64 (java.util package) to encode encrypted data and decode it back, then display results.

Java code for DES

```
import javax.crypto.Cipher;
import javax.crypto.SecretKey;
import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.DESKeySpec;
import java.util.Base64;
import java.util.Scanner;

public class desR {

public static void main(String[] args) {
try {
Scanner scanner = new Scanner(System.in);
```

```
System.out.print("Enter 8-character key: ");
String keyString = scanner.nextLine();

System.out.print("Enter plaintext: ");
String plaintext = scanner.nextLine();

DESKeySpec desKeySpec = new DESKeySpec(keyString.getBytes());
SecretKeyFactory keyFactory = SecretKeyFactory.getInstance("DES");
SecretKey secretKey = keyFactory.generateSecret(desKeySpec);

Cipher cipher = Cipher.getInstance("DES/ECB/PKCS5Padding");

cipher.init(Cipher.ENCRYPT_MODE, secretKey);
byte[] encryptedBytes = cipher.doFinal(plaintext.getBytes());
String encryptedText = Base64.getEncoder().encodeToString(encryptedBytes);

System.out.println("\nEncrypted (Base64): " + encryptedText);

cipher.init(Cipher.DECRYPT_MODE, secretKey);
byte[] decryptedBytes = cipher.doFinal(Base64.getDecoder().decode(encryptedText));
String decryptedText = new String(decryptedBytes);

System.out.println("Decrypted Text: " + decryptedText);

} catch (Exception e) {
e.printStackTrace();
}
}
}
```

COMPILE AND RUN THE PROGRAM

```
>javac desR.java
>java desR
```

OUTPUT:DES

```
Enter 8-character key: lloworld
Enter plaintext: Maharaja

Encrypted (Base64): xf6JDpwW4KVraMq2o2wZ+w==
Decrypted Text: Maharaja
```

Result :

Encryption and Decryption using Data Encryption Standard process is successfully executed.

Ex.no: 03	Advanced Encryption Standard
10.02.2026	

Aim: Develop a Program to do Encryption and Decryption using Advanced Encryption Standard

Advanced Encryption Standard

The Advanced Encryption Standard (AES) is a symmetric key encryption algorithm used to securely convert plain text into cipher text for protecting sensitive data.

AES works by:

1. Dividing data into 128-bit blocks
2. Performing multiple rounds (10, 12, or 14 depending on key size)
3. Applying operations like:
    - Substitution (SubBytes)
    - Row shifting (ShiftRows)
    - Column mixing (MixColumns)
    - Key addition (AddRoundKey)
4. Producing secure encrypted data (cipher text)

Procedure:

1. Define the class, methods, and main function to control AES encryption and decryption flow.
2. Use Scanner (java.util package) to take user choice, key, and input text.
3. Generate a 128-bit AES key using SecretKeySpec (javax.crypto.spec package) from the given key string.
4. Use Cipher (javax.crypto package) with AES algorithm to initialize encryption and decryption modes.
5. Perform encryption/decryption using doFinal() method on input byte data.
6. Use Base64 (java.util package) to encode encrypted bytes and decode them back for readable output.

Java code for AES

```
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;
import java.util.Scanner;

public class AES {

    static SecretKeySpec getKey(String key) throws Exception {
        byte[] k = key.getBytes();
        byte[] keyBytes = new byte[16]; // 128-bit key

        for (int i = 0; i < k.length && i < 16; i++) {
            keyBytes[i] = k[i];
        }

        return new SecretKeySpec(keyBytes, "AES");
    }

    static String encrypt(String text, String key) throws Exception {
        Cipher c = Cipher.getInstance("AES");
        c.init(Cipher.ENCRYPT_MODE, getKey(key));
        byte[] enc = c.doFinal(text.getBytes());
        return Base64.getEncoder().encodeToString(enc);
    }

    static String decrypt(String text, String key) throws Exception {
        Cipher c = Cipher.getInstance("AES");
        c.init(Cipher.DECRYPT_MODE, getKey(key));
        byte[] dec = Base64.getDecoder().decode(text);
        return new String(c.doFinal(dec));
    }

    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter e/d: ");
        String ch = sc.nextLine();

        System.out.print("Enter Key: ");
        String key = sc.nextLine();

        if (ch.equalsIgnoreCase("e")) {
            System.out.print("Enter Text: ");
            String text = sc.nextLine();
            System.out.println("Encrypted: " + encrypt(text, key));
```

```
        } else {
            System.out.print("Enter Encrypted Text: ");
            String text = sc.nextLine();
            System.out.println("Decrypted: " + decrypt(text, key));
        }
    }
}
```

COMPILE AND RUN THE PROGRAM

```
>javac AES.java
>java AES
```

OUTPUT:AES

Encryption:
Enter e/d: e
Enter Key: 741582
Enter Text: King of Oceans
Encrypted: PQhu0jrJiYLy5QC9EZiMzw==
Decryption:
Enter e/d: d
Enter Key: 741582
Enter Encrypted Text: PQhu0jrJiYLy5QC9EZiMzw==
Decrypted: King of Oceans

Result :

Encryption and Decryption using Advanced Encryption Standard process is successfully executed.

Ex.no:04	Rivest Shamir Adleman Algorithm (RSA)
10.02.2026	

Aim: Develop a Text of 30 Words, Perform Encryption and Decryption using RSA

Rivest–Shamir–Adleman Algorithm

The Rivest–Shamir–Adleman (RSA) algorithm is an asymmetric key encryption algorithm used to securely transmit data by using a pair of keys: a public key and a private key.

1. Two large prime numbers are selected
2. Keys (public and private) are generated
3. Sender encrypts data using the **receiver's public key**
4. Receiver decrypts data using their **private key**

Procedure:

1. Define the class, helper methods (gcd, modPow), and main method to implement RSA algorithm flow.
2. Use Scanner (java.util package) to read the input message from the user.
3. Initialize prime numbers p and q, then compute n = p*q and phi = (p−1)*(q−1).
4. Find public key e using gcd() and compute private key d such that (e*d) % phi = 1.
5. Perform encryption using modPow(message, e, n) and store results in a cipher array.
6. Perform decryption using modPow(cipher, d, n) and convert numeric values back to characters.

Java code for RSA

```
import java.util.Scanner;

public class RSA {

   static int gcd(int a, int b)
   {
      if(b==0)
         return a;
      else
         return gcd(b,a%b);
   }

   static long modPow(long base,long exp,long mod)
   {
      long result = 1;
```

```
    for(int i=0;i<exp;i++)
    {
        result = (result * base) % mod;
    }

    return result;
}

public static void main(String args[])
{
    int p=61,q=53;
    int n,phi,e=2,d=0;

    n = p*q;
    phi = (p-1)*(q-1);

    for(int i=2;i<phi;i++)
    {
        if(gcd(i,phi)==1)
        {
            e=i;
            break;
        }
    }

    for(int i=1;i<phi;i++)
    {
        if((e*i)%phi==1)
        {
            d=i;
            break;
        }
    }

    System.out.println("Public Key (e,n): "+e+","+n);
    System.out.println("Private Key (d,n): "+d+","+n);

    Scanner sc=new Scanner(System.in);

    System.out.println("\nEnter 30 words message:");
    String msg=sc.nextLine();

    long cipher[]=new long[msg.length()];

    System.out.println("\nEncrypted Message:");
    for(int i=0;i<msg.length();i++)
    {
        int t = msg.charAt(i);
```

```
            cipher[i] = modPow(t,e,n);
            System.out.print(cipher[i]+" ");
        }

        System.out.println("\n\nDecrypted Message:");
        for(int i=0;i<msg.length();i++)
        {
            long ch = modPow(cipher[i],d,n);
            System.out.print((char)ch);
        }

    }
}
```

COMPILE AND RUN THE PROGRAM

```
>javac RSA.java
>java RSA
```

OUTPUT:RSA

```
Public Key (e,n): 7,3233
Private Key (d,n): 1783,3233

Enter 30 words message:
Bharathidasan University of College is the best college in Trichy

Encrypted Message:
2241 3052 1818 1797 1818 1762 3052 3020 2872 1818 567 1818 1544 2774 2509 1544 3020
1794 3071 1797 567 3020 1762 731 2774 3183 576 2774 373 3183 1877 1877 3071 728
3071 2774 3020 567 2774 1762 3052 3071 2774 2920 3071 567 1762 2774 24 3183 1877
1877 3071 728 3071 2774 3020 1544 2774 1623 1797 3020 24 3052 731 2774

Decrypted Message:
Bharathidasan University of College is the best college in Trichy
```

Result :

Encryption and Decryption using Rivest–Shamir–Adleman (RSA) algorithm process is successfully executed.

Ex.no:05	Blowfish Algorithm
13.02.2026	

Aim: Develop a Program to do Encryption and Decryption using Blowfish Algorithm

Blowfish Algorithm

The Blowfish algorithm is a symmetric key block cipher used for encrypting data by converting plain text into cipher text using a secret key.

Blowfish works by:

1. Dividing data into 64-bit blocks
2. Using a variable-length secret key
3. Performing 16 rounds of encryption using substitution and permutation
4. Producing encrypted output (cipher text)

Procedure:

1. Define the class and main method to control Blowfish encryption and decryption process.
2. Use Scanner (java.util package) to take plaintext input from the user.
3. Generate a secret key using KeyGenerator and SecretKey (javax.crypto package) for Blowfish algorithm.
4. Create and initialize Cipher (javax.crypto package) with Blowfish in encryption mode.
5. Perform encryption and decryption using doFinal() method on byte data.
6. Use Base64 (java.util package) to encode encrypted output and decode it back for readable decrypted text.

Java code for Blowfish Algorithm

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import java.util.Base64;
import java.util.Scanner;

public class Blowfish {
public static void main(String[] args) throws Exception {

Scanner sc = new Scanner(System.in);

System.out.print("Enter the text: ");
String plainText = sc.nextLine();
```

```
KeyGenerator keyGen = KeyGenerator.getInstance("Blowfish");
SecretKey key = keyGen.generateKey();

Cipher cipher = Cipher.getInstance("Blowfish");
cipher.init(Cipher.ENCRYPT_MODE, key);

byte[] encryptedBytes = cipher.doFinal(plainText.getBytes());
String encryptedText = Base64.getEncoder().encodeToString(encryptedBytes);

System.out.println("Encrypted Text: " + encryptedText);

cipher.init(Cipher.DECRYPT_MODE, key);

byte[] decryptedBytes = cipher.doFinal(Base64.getDecoder().decode(encryptedText));
String decryptedText = new String(decryptedBytes);

System.out.println("Decrypted Text: " + decryptedText);
}
}
```

COMPILE AND RUN THE PROGRAM

```
>javac Blowfish.java
>java Blowfish
```

OUTPUT:Blowfish

```
Enter the text: Premkumar
Encrypted Text: NH3dw9TyneOWGOibMXvh1Q==
Decrypted Text: Premkumar
```

Result :

Encryption and Decryption using Blowfish algorithm process is successfully executed.

Ex.no:06	Vigenere Cipher
20.02.2026	

Aim: Develop a Program to do Encryption and Decryption using Vigenere Cipher

Vigenere Cipher

The Vigenère Cipher is a classical symmetric encryption technique that uses a keyword to encrypt plain text by applying a series of Caesar shifts based on the letters of the keyword.

Working:

1. Choose a keyword (e.g., KEY)
2. Repeat the keyword to match the length of the message
3. Each letter in the plain text is shifted according to the corresponding letter in the keyword
4. Generate the cipher text

Procedure:

1. Define the class and main method to control the Vigenère Cipher execution flow.
2. Use Scanner (java.util package) to take plaintext and key input from the user.
3. Convert the key into shift values using character positions (key.charAt − 'a').
4. Perform encryption using (text + key shift) % 26 with cyclic key repetition.
5. Perform decryption using (cipher − key shift + 26) % 26 to recover original text.
6. Use string concatenation and loops to process characters and display encrypted and decrypted results.

Java Code for Vigenere Cipher

```
import java.util.Scanner;
public class VigenereCipher {
public static void main(String[] args) {
Scanner sc = new Scanner(System.in);
System.out.print("Enter Plain Text (lowercase): ");
String text = sc.nextLine();
System.out.print("Enter Key: ");
String key = sc.nextLine();
String cipher = "";
int j = 0;
for (int i = 0; i < text.length(); i++) {
char ch = text.charAt(i);
int shift = key.charAt(j % key.length()) - 'a';
cipher += (char)((ch - 'a' + shift) % 26 + 'a');
```

```
j++;
}
System.out.println("Encrypted Text: " + cipher);
String plain = "";
j = 0;
for (int i = 0; i < cipher.length(); i++) {
char ch = cipher.charAt(i);
int shift = key.charAt(j % key.length()) - 'a';
plain += (char)((ch - 'a' - shift + 26) % 26 + 'a');
j++;
}
System.out.println("Decrypted Text: " + plain);
}}
```

COMPILE AND RUN THE PROGRAM

```
>javac VigenereCipher.java
>java VigenereCipher
```

OUTPUT: VIGENERE CIPHER

Enter Plain Text (lowercase): bharathidasan university

Enter Key: HOD

Encrypted Text: IV^YOWOWaHa^U\XUWYL`VPb\

Decrypted Text: bharathidasannuniversity

Result :

Encryption and Decryption using Vigenere Cipher process is successfully executed.

Ex.no:07	Digital Signature
20.02.2026	

Aim: Develop a Program to do Encryption and Decryption using Digital Signature

Digital Signature

A Digital Signature is a cryptographic technique used to verify the authenticity and integrity of a digital message, document, or data using public key cryptography.

1. Sender creates a hash (digest) of the message
2. The hash is encrypted using the sender's private key → this is the digital signature
3. The receiver decrypts the signature using the sender's public key
4. The receiver also generates a hash of the received message
5. If both hashes match → the message is authentic and unchanged

Procedure:

1. Define the class and main method to handle digital signature creation and verification flow.
2. Use Scanner (java.util package) to read the input message from the user.
3. Generate key pair using KeyPairGenerator, PrivateKey, PublicKey (java.security package) for DSA algorithm.
4. Create Signature object (java.security package) with SHA256withDSA and initialize with private key for signing.
5. Perform digital signing using update() and sign(), then encode using Base64 (java.util package).
6. Verify the signature using public key and verify() method to check message authenticity.

Java Code for Digital Signature

```
import java.security.*;
import java.util.Base64;
import java.util.Scanner;

public class DSADigitalSignature {

    public static void main(String[] args) throws Exception {

        Scanner sc = new Scanner(System.in);

        System.out.print("Enter Message: ");
        String message = sc.nextLine();
```

```
        KeyPairGenerator keyGen = KeyPairGenerator.getInstance("DSA");
        keyGen.initialize(2048);
        KeyPair pair = keyGen.generateKeyPair();

        PrivateKey privateKey = pair.getPrivate();
        PublicKey publicKey = pair.getPublic();

        Signature sign = Signature.getInstance("SHA256withDSA");
        sign.initSign(privateKey);
        sign.update(message.getBytes());

        byte[] signatureBytes = sign.sign();
        String signature = Base64.getEncoder().encodeToString(signatureBytes);

        System.out.println("Digital Signature: " + signature);

        Signature verify = Signature.getInstance("SHA256withDSA");
        verify.initVerify(publicKey);
        verify.update(message.getBytes());

        boolean isValid = verify.verify(Base64.getDecoder().decode(signature));

        if (isValid)
            System.out.println("Signature Verified (Valid)");
        else
            System.out.println("Signature Verification Failed");

        sc.close();
    }
}
```

COMPILE AND RUN THE PROGRAM

```
>javac DigitalSignature.java
>java DigitalSignature
```

OUTPUT: DIGITALSIGNATURE

Enter Message: prem

Digital Signature:
MDwCHHJocv4oAR3Q5yOSSrggr6XF9rgZPe4R+A16MJMCHCrlaaYqGg99/m36fKcwPh2
yvbQdc1KzH31Aqu0=

Signature Verified (Valid)

Result :

Encryption and Decryption using Digital Signature process is successfully executed.

Ex.no:08	Substitution Cipher
24.02.2026	

Aim: Develop a Given Text, Perform Encryption and Decryption using Substitution Cipher

Substitution Cipher

A Substitution Cipher is a classical encryption method where each letter in the plain text is replaced by another letter, number, or symbol using a fixed substitution rule (key).

1. Define a **substitution key** (e.g., A→X, B→M, C→P...)
2. Replace every letter in the message using this rule
3. Produce the **cipher text**

Procedure:

1. Define the class, methods (encrypt, decrypt, isValidKey), and main method to control substitution cipher flow.
2. Use Scanner (java.util package) to take key and plaintext input from the user.
3. Validate the key (26 unique letters) using isValidKey() to ensure proper substitution mapping.
4. Perform encryption by replacing each alphabet using key index mapping from the original alphabet.
5. Perform decryption by finding index of cipher character in key and mapping back to original alphabet.
6. Use loops and string operations to process characters and display encrypted and decrypted results.

Java Code for Substitution Cipher

```
import java.util.Scanner;

public class SubstitutionCipher {

    static String alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ";

    // Encryption
    public static String encrypt(String plainText, String key) {
        plainText = plainText.toUpperCase();
        String cipherText = "";

        for (int i = 0; i < plainText.length(); i++) {
            char ch = plainText.charAt(i);
```

```
        if (Character.isLetter(ch)) {
            int index = alphabet.indexOf(ch);
            cipherText += key.charAt(index);
        } else {
            cipherText += ch;
        }
    }
    return cipherText;
}

// Decryption
public static String decrypt(String cipherText, String key) {
    cipherText = cipherText.toUpperCase();
    String plainText = "";

    for (int i = 0; i < cipherText.length(); i++) {
        char ch = cipherText.charAt(i);

        if (Character.isLetter(ch)) {
            int index = key.indexOf(ch);
            plainText += alphabet.charAt(index);
        } else {
            plainText += ch;
        }
    }
    return plainText;
}

// Validate key
public static boolean isValidKey(String key) {
    key = key.toUpperCase();

    if (key.length() != 26) return false;

    for (char ch : alphabet.toCharArray()) {
        if (key.indexOf(ch) == -1) {
            return false;
        }
    }
    return true;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    System.out.print("Enter substitution key (26 unique letters): ");
    String key = sc.nextLine().toUpperCase();
```

```
        if (!isValidKey(key)) {
            System.out.println("Invalid key! Must contain all 26 letters exactly once.");
            return;
        }

        System.out.print("Enter plaintext: ");
        String plainText = sc.nextLine();

        String encrypted = encrypt(plainText, key);
        System.out.println("Encrypted text: " + encrypted);

        String decrypted = decrypt(encrypted, key);
        System.out.println("Decrypted text: " + decrypted);

        sc.close();
    }
}
```

COMPILE AND RUN THE PROGRAM

```
>javac SubstitutionCipher.java
>java SubstitutionCipher
```

OUTPUT: SUBSTITUTION CIPHER

Enter substitution key (26 unique letters): qwertyuiopasdfghjklzxcvbnm

Enter plaintext: Premkumar

Encrypted text: HKTDAXDQK

Decrypted text: PREMKUMAR

Result :

Encryption and Decryption using Substitution Cipher process is successfully executed.

Ex.no:09	Web Security
24.02.2026	

Aim: Develop a Given Numerical and Textual Data, Perform Web Security

Web Security

Web Security refers to the measures, technologies, and practices used to protect websites, web applications, and online services from cyber threats, attacks, and unauthorized access.

Procedure:

1. Define the class, hashing method, and main method to manage web security operations.
2. Use Scanner (java.util package) to take user choice, username, and password input.
3. Use MessageDigest (java.security package) to generate SHA-256 hash of the password.
4. Use Base64 (java.util package) to encode hashed password into readable format.
5. Store user credentials using HashMap (java.util package) for username and hashed password mapping.
6. Perform registration and login verification by comparing stored hash with entered password hash.

Java Code for Web Security

```
import java.security.MessageDigest;
import java.util.Base64;
import java.util.HashMap;
import java.util.Scanner;

public class WebSecurity {

    public static String hashSHA256(String data) throws Exception {
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        byte[] hash = digest.digest(data.getBytes("UTF-8"));
        return Base64.getEncoder().encodeToString(hash);
    }

    public static void main(String[] args) throws Exception {
        Scanner sc = new Scanner(System.in);
        HashMap<String, String> userDB = new HashMap<>(); // Store username -> hashed
password
```

```
        while (true) {
            System.out.print("\nChoose option (1: Register / 2: Login / 0: Exit): ");
            int choice = Integer.parseInt(sc.nextLine());

            if (choice == 0) break;

            System.out.print("Enter username: ");
            String username = sc.nextLine();

            System.out.print("Enter password: ");
            String password = sc.nextLine();

            String hashedPassword = hashSHA256(password);

            if (choice == 1) { // Register
                userDB.put(username, hashedPassword);
                System.out.println("User registered successfully ");
            } else if (choice == 2) { // Login
                if (userDB.containsKey(username) &&
userDB.get(username).equals(hashedPassword)) {
                    System.out.println("Authentication Success ");
                } else {
                    System.out.println("Authentication Failed");
                }
            } else {
                System.out.println("Invalid option.");
            }
        }

        sc.close();
        System.out.println("Exiting...");
    }
}
```

COMPILE AND RUN THE PROGRAM

```
>javac WebSecurity.java
>java WebSecurity
```

OUTPUT:WEB SECURITY

Choose option (1: Register / 2: Login / 0: Exit): 1
Enter username: Premkumar
Enter password: 0519
User registered successfully

Choose option (1: Register / 2: Login / 0: Exit): 2
Enter username: Premkumar
Enter password: 0519
Authentication Success

Result :

Numerical and Textual Data, Perform Web Security process is successfully executed.